A Convention for Explicit Declaration of Environments and Top-Down Refinement of Data

EDWIN TOWSTER, MEMBER, IEEE

Abstract—**Many problems of block structure derive from the remote specification of parts of the local environment. An alternative to block structure is proposed, in which the local environment is described within each program module. Concise environment descriptions are made possible by creating a tree whose terminal nodes are declarations and whose intermediate nodes can represent all of their descendant terminal node declarations. The tree encompasses all of the data used by the program and can thus be thought of as the data structure for the entire program. Each intermediate node of the tree names an abstract data structure that is implemented from the data named in its descendant nodes. Top-down refinement of abstract data structures consists of creating descendant nodes, and terminates when all terminal nodes of the tree have been created. This activity occurs as a normal part of program development, along with the top-down refinement of program modules. The tree can be used in place of a symbol table as a referencing structure for compilation or interpretation. Compilation time will then increase linearly with the length of the program and the referencing structure can grow dynamically during compilation.**

Index Terms—**Abstract data, block structure, chunk structure, compilation time, locality of reference, naming structure, programming psychology, refinement of data, scope rules, structured programming.**

INTRODUCTION

PROBLEMS and limitations related to block structure, scope of variables, and control of naming environments in programming languages have been examined by many authors [1]-[8]. Wulf and Shaw [2] describe the problems of block structure as follows.

1) Side effects of procedures.

2) Indiscriminate access, i.e., the inability to allow access to procedures which implement a data structure, while denying access to the private data used in the implementation.

3) Vulnerability, i.e., the possibility of (inadvertently) interposing a conflicting declaration between a variable used in an inner block and its declaration in a distant outer block.

4) No overlapping definitions. Suppose that in a sequence of code segments each segment uses just the output data of the immediately preceding segment. Ideally, every item of data would then be available to just those two successive segments that use it. However, since block structure does not permit blocks to overlap, the data must be declared for the entire sequence of segments, thereby forcing segments to have access to data that they do not use.

Manuscript received October 25, 1977; revised November 17, 1978.

The author is with the Department of Computer Science, University of Southwestern Louisiana, Lafayette, LA 70504.

5) Situations in which the interpretation or use of a variable is hard to understand. Wulf and Shaw specifically mention the "hole-in-the-scope" problem [9], [10], i.e., the problem of whether a global variable occurring within a procedure body should be (statically) bound to a declaration whose scope includes the procedure body or (dynamically) bound to a declaration whose scope includes a call to the procedure. Both intepretations are possible under block structure, and both are used in existing programming languages.

Parnas [6] additionally mentions:

6) the inability to reenter a previously exited environment.

We also note, with respect to the human element in programming:

7) it is not easy to keep track of one's current environment in a "heavily blocked" program, so that there is an incentive to use fewer blocks, thereby making variables more global than necessary.

Wulf and Shaw suggest that the textual representation of access to a name should be explicit and localized. In fact, most of the above problems could be resolved by a textual representation that explicitly describes the local environment and allows chosen parts of it to be passed to dependent sections of code. The apparent difficulty with this suggestion is that the complete description of an environment would ordinarily contain too much text to be conveniently listed in every locality.

In order to describe an environment concisely, we suggest that declarations that are conceptually related be grouped together (cf. [11]) and given a name that describes and stands for the entire group. These names can in turn be organized into conceptual groups at a higher level and the process continued until only a small number of named groups exist. An environment can then be described by listing just those names which exactly "cover" the intended declarations. If the groups have been well chosen with respect to the data usage of the program, then relatively few names will be needed to describe local environments.

We do not actually propose to proceed in this "bottom-up" approach to data organization, but instead suggest using a "top-down" approach, as described in the next section.

CHUNK STRUCTURE

Let us assume that procedures are coded top-downwards into modular units limited to a fixed length, in the manner described by Mills [12]. This length limit might, for example, be one page of code (about fifty lines) per module.

As the program is being coded a "chunk tree" describing the

0098-5589/79/0700-0374$00.75

```
1   begin "Implement a simple macro insertion facility. Read implementation
2         restrictions and macros from <input_file. Output the macro-expanded
3         text of the main program (the first macro) to output_file< or output
4         an error message to error_output< file.";
5
6   env;
7   <input_file stream input file
8   output_file< stream output file
9   /implementation_restrictions
10        max_#_macros fixed binary
11        max_length_source_deck fixed binary
12        max_depth_macro_nesting fixed binary
13  /macro_storage
14  /error_control
15        no_error bit(1)
16        error_output< stream output file
17  end env;
18
19  open file(error_output);
20  no_error = '1'b;
21  get file(input_file) list(implementation_restrictions);
22  "Read the macros from <input_file into macro_storage<. Observe
23        <implementation_restrictions. Maintain <error_control<."
24        /* Fig. 1B */;
25  if    no_error
26  then "Macro expand the main program from the macros in <macro_storage and
27             output the resulting text to output_file<. Observe
28             <implementation_restrictions. Maintain <error_control<."
29             /* Fig. 1D */;
30  else /* do nothing */;
31  close file(error_output);
32  end;
```

(a)

Fig. 1(a). Sample program module "insert_A" (top module).

data structure of the program is formed. ("Chunk" is a term used in psychology. Its use here will be justified later.) Data declarations will be located at the terminal nodes of the chunk tree. Every node of the tree is a "chunk" with a "chunk name" that stands for the declarations at the terminal nodes of its subtree. The programmer need not conceive of the terminal nodes before coding begins, only the topmost nodes of the tree, as needed to functionally describe the data in the topmost modules of the program. Additional nodes are created and attached to the tree as programming proceeds, with descendant nodes representing the data used in implementing the data structure described by their ancestor nodes. The tree is thus developed top-downwards (from root to terminal nodes) as the program is being written.

The programmer coding a new module can work from a reference listing of the current version of the chunk tree. The listing should place all direct descendants of a node on the same page, where they can be viewed together. This sets a limit of about fifty direct descendants for any node in the chunk tree. Since it is advantageous to conceptualize data into fairly small-sized groups, this limit should not be perceived by the programmer as a restrictive condition. The direct descendants of a node are usually not all created at the same time, but instead each is created as the need for its arises.

Each module of the program is headed by a title that describes what the module is intended to do (see Fig. 1(a), lines 1-4). The entire environment that is being passed to the module is designated by the chunk names that appear in the title. A special "chunk mark" (a small arrow "←") is used to distinguish chunk names from the other text of the title and also to indicate whether the part of the environment designated by the chunk name is input to the module, output from the module, or both. Input and output are indicated by prefixing or suffixing (respectively) the chunk mark to the chunk name. For example, the title (Fig. 1(a), lines 22-23 and Fig. 1(b), lines 1-2)

> Read the macros from ←input_file into macro_storage←. Observe ←implementation_restrictions. Maintain ←error_control←.

says what the module is supposed to do and indicates that the environment for the module is given by the terminal node declarations that will eventually be descendants of input_file, macro_storage, implementation_restrictions, and error_control. Input data for the module is given by input_file, implementation_restrictions, and error_control. Output data from the module is given by macro_storage and error_control.

The title approximates a functional specification for the module, with functional domain and range corresponding to module input and output, respectively, and the title giving (in an imperative form) a rule of correspondence for the function [13], [14]. Passing data as "input-only" to a submodule implies that the submodule does not change the output value of the data. Passing data as "output-only" implies that the submodule has the responsibility for initializing the data or must pass on this responsibility (cf. Dijkstra's "virgin variable" [7]). Chunk names occurring in the title of the topmost module of a procedure describe the external environment that is being passed to the procedure.

Following the title, the module contains a section used for describing its environment (Fig. 1(a), lines 6-17). The environment description section is delimited by "env" and "end env" statements. Chunk names occurring in the title are repeated in this section and, depending on the needs of the module, possibly refined by attaching descendant nodes. These nodes may be newly defined or may have previously occurred in the environment description section of some other module. A

```
1   begin "Read the macros from <input_file into macro_storage<. Observe
2         <implementation_restrictions. Maintain <error_control<.";
3
4   env;
5   <input_file stream input file
6   <implementation_restrictions
7        max_#_macros fixed binary
8        max_length_source_deck fixed binary
9   macro_storage<
10       macro_directory
11            #_macros fixed binary initial(0)
12            macro(max_#_macros)
13                 name char(8)
14                 location fixed binary
15       macro_definition_table
16            /length fixed binary initial(0)
17            line(max_length_source_deck) char(80)
18       /* Assert: #_macros is the number of macros that have been read from
19            input_file. Macro(i) is the i-th macro read, i = 1 to #_macros.
20            The i-th macro body is stored in macro_definition_table from
21            line(macro(i).location) to one line before the next line =
22            'end of macro'||(68)' '. The first macro is the main program,
23            with name = 'main_pgm'. Length specifies the current extent of
24            the macro_definition_table. */
25  <error_control<
26       no_error bit(1)
27       error_output stream output file
28  /card_image char(80)
29  end env;
30
31  get file(input_file) edit(card_image)(column(1), a(80));
32  do while(no_error  &  substr(card_image,1,18) ^= 'end of source deck');
33       if   #_macros + 1 > max_#_macros
34       then do;
35            no_error = '0'b;
36            put file(error_output) list('Too many macros');
37            end;
38       else do;
39            #_macros = #_macros + 1;
40            macro.name(#_macros) = substr(card_image,14,8);
41            macro.location(#_macros) = macro_definition_table.length + 1;
42            "Read the next macro body from <input_file into the
43                 <macro_definition_table<, leaving card_image< at the next
44                 card after the macro body. Observe
45                 <implementation_restrictions. Maintain <error_control<."
46                 /* Fig. 1C */;
47            end;
48       end;
49  end;
```

(b)

Fig. 1(b). Sample program module "insert_B."

chunk name can also be refined by attributing a type to it, thereby making it a terminal node of the chunk tree. For example, the node macro_storage might eventually be refined to

```
macro_storage
     macro_directory
     macro_definition_table
```

while the node input_file might be refined to

```
input_file stream input file.
```

Descendant nodes are indicated by indentation, as described by Knuth [15]. Thus macro_directory and macro_definition_table (above) are direct descendants of macro_storage. These chunk tree refinements are pictured in Fig. 2.

Some data may be local to the module and are so indicated by listing their chunk names in the environment description section with a slash prefixed to it (Fig. 1(a), lines 9, 13, 14). These local chunk names may be attached as descendant nodes to an existing node (Fig. 1(b), line 16) or may be listed independently (as in Fig. 1(a), line 9), depending on which use seems conceptually most natural. In order to maintain a single chunk tree, independent local nodes must be attached to nodes that are already in the tree. Nodes for the modules are incorporated into the chunk tree for this purpose. Nodes for submodules of a module are attached as descendant nodes to the module's node. The module hierarchy tree is thus embedded within the chunk tree. Each independent local node is attached to the module node to which it is local. There is an additional node to which all external data as well as the top module of the program are attached. This "external" node is the root node of the chunk tree. The chunk tree thus contains all program references while having limited branching from any one node. The chunk tree as constructed from the module of Fig. 1(a) is pictured in Fig. 3. For simplicity we have labeled each module with a name (e.g., "insert_A") rather than its title.

Intermediate nodes of the chunk tree can take attributes, such as dimensions or storage class designators (Fig. 1(b), line 12). An attribute of data allocated within a module can depend on an expression involving values of input-only data passed to the module. The actual module or submodule in which local data is allocated depends on the allocation strategy used by the system.

A module that is to be referenced by several modules should be listed as data of type "entry." It can then be passed (as a chunk) to the modules that reference it. The module title

```
begin "Read the next macro body from ‹input_file into the
       ‹macro_definition_table‹, leaving card_image‹ at the next card after
       the macro body. Observe ‹implementation_restrictions. Maintain
       ‹error_control‹.";

env;
‹input_file stream input file
‹implementation_restrictions
     max_length_source_deck fixed binary
‹macro_definition_table‹
     length fixed binary
     line(max_length_source_deck) char(80)
     /* Assert: The i-th macro body is stored in macro_definition_table from
          line(macro(i).location) to one line before the next line =
          'end of macro'||(68)' '. Length specifies the current extent of the
          macro_definition_table. */
card_image‹ char(80)
‹error_control‹
     no_error bit(1)
     error_output stream output file
end env;

get file(input_file) edit(card_image)(column(1), a(80));
do while(no_error  &  substr(card_image,1,13) ^= 'define macro '
                   &  substr(card_image,1,18) ^= 'end of source deck');
     if   length + 1 > max_length_source_deck
     then do;
          no_error = '0'b;
          put file(error_output) list('Deck too long');
          end;
     else do;
          length = length + 1;
          macro_definition_table.line(length) = card_image;
          get file(input_file) edit(card_image)(column(1), a(80));
          end;
     end;
if   no_error
then if   length + 1 > max_length_source_deck
     then do;
          no_error = '0'b;
          put file(error_output) list('Deck too long');
          end;
     else do;
          length = length + 1;
          macro_definition_table.line(length) = 'end of macro';
          end;
else /* do nothing */;
end;
```

(c)

Fig. 1(c). Sample program module "insert_C."

appears with the entry attribute, as in the following example:

push_A entry ("Push ←x onto ←stack_A←.")

At times it is useful to restructure the chunk tree for the benefit of a particular module and its submodules. This might be done to maintain concise environment descriptions or because the restructured tree seems conceptually more natural for the particular module environment. Restructuring is accomplished by listing chunk names from the title as descendant nodes to a local node (Fig. 1(a), line 16). These chunk names are distinguished from newly-created descendant nodes by their chunk marks, as retained from the title.

The environment description section is followed by executable code (Fig. 1(a), lines 19-31). Names used in the code can only refer to the part of the chunk tree listed in the environment description section, i.e., every name occurring in executable code must appear in the environment description section of the module. The scope of a chunk name thus consists of all modules that list the name in their environment description sections. To avoid ambiguity, a name occurring more than once in the environment description section appears in executable code as a qualified name, following the rules for PL/I structures (Fig. 1(d), "macro," "name," "location," and "line").

The executable code may reference submodules by listing their titles within quotes (Fig. 1(a), lines 22-23, lines 26-28). Chunk names used within such a title must appear in the environment description section and will indicate the part of the module environment that is being passed to the referenced submodule. The title thus acts as the total interface between the module and the submodule, showing both the data passed to the submodule and what the submodule is expected to do with it. A referenced submodule might be implemented by the system as a subroutine or as a macro. If implemented as a subroutine, then chunk names in its title act as arguments to the subroutine. If implemented as a macro, then chunk names mentioned in the title are not actually "passed" to the macro, but each chunk name in the macro is in effect replaced by its fully qualified name in the chunk tree. Rules for identifying the titles of function procedures are described in [16].

A chunk name may be used as an operand, in which case the operation is applied to all of the data described by the terminal nodes of its subtree (Fig. 1(a), line 21). Operations that have meaningful interpretations when thus applied include: assignment (between subtrees of the same "tree type"), test of equality, input/output, and allocate/free (for based subtrees).

```
begin "Macro expand the main program from the macros in <macro_storage and
       output the resulting text to output_file<. Observe
       <implementation_restrictions. Maintain <error_control<.";

env;
output_file< stream output file
<implementation_restrictions
<macro_storage
     macro_directory
          macro(max_#_macros)
               name char(8)
               location fixed binary
     macro_definition_table
          line(max_length_source_deck) char(80)
     /* Assert: The i-th macro body is stored in macro_definition_table from
          line(macro(i).location) to one line before the next line =
          'end of macro'||(68)' '. The first macro is the main program, with
          name = 'main_pgm'. */
<error_control<
     no_error bit(1)
/current_position
     macro
          name char(8)
          location fixed binary
     line
          image char(80)
          reference_point fixed binary
/macro_reference_stack
/end_of_main_program bit(1)
end env;

open file(output_file);
current_position.macro = macro_directory.macro(1);
end_of_main_program = '0'b;
"Set the macro_reference_stack< empty.";
do while(no_error & ~end_of_main_program);
     image = macro_definition_table.line(current_position.location);
     reference_point = index(image, '%include ');
     /* case */;
          if   reference_point = 0  &  image ~= 'end of macro'||(68)' '
          then do;
               put file(output_file) edit(image)(skip,a);
               current_position.location = current_position.location + 1;
               end;
     else if   reference_point ~= 0
          then do;
               put file(output_file) edit(substr(image,1, reference_point-1)
                                          (skip,a);
               "Push <current_position onto the <macro_reference_stack<. Observe
                    <implementation_restrictions. Maintain <error_control<.";
               if   no_error
               then "Set current_position.macro< to the macro with name
                         <(substr(image, reference_point+9, 8)), as found in the
                         <macro_directory. Maintain <error_control<.";
                         /* Note: <( ... ) will be passed as a dummy
                              argument */;
               else /* do nothing */;
               end;
     else if   reference_point = 0  &  image = 'end of macro'||(68)' '
          then if   current_position.macro.name = 'main_pgm'
               then end_of_main_program = '1'b;
               else do;
                    "Pop current_position< from the <macro_reference_stack<.";
                    put file(output_file)
                         edit(substr(image, reference_point+17)(skip,a);
                    current_position.location = current_position.location + 1;
                    end;
     /* end case */;
     end;
close file(output_file);
end;
```

(d)

Fig. 1(d). Sample program module "insert_D."

An Example

These conventions are illustrated in Fig. 1 by several modules for a simple macro insertion facility that allows nested macro references but does not allow nested (dynamically created) macro definitions [17]. A complete version of the program is given in [16].

With respect to a given module, for example the Fig. 1(b) module, several categories of chunks can be distinguished. Some chunks are not passed to the module (output_file). Some chunks are explicitly passed to the module (input_file, macro_storage, implementation_restrictions, and error_control). Some are not explicitly passed, but belong to the subtree of an explicitly passed chunk (e.g., macro_directory and no_error). These implicitly passed data are described by their explicitly passed ancestral chunks in the sense that any well-chosen chunk name describes the function of its subtree. Some of the implicitly passed chunks may be refinements that are made for the first time within the module (e.g., macro_directory). These chunks might actually be allocated

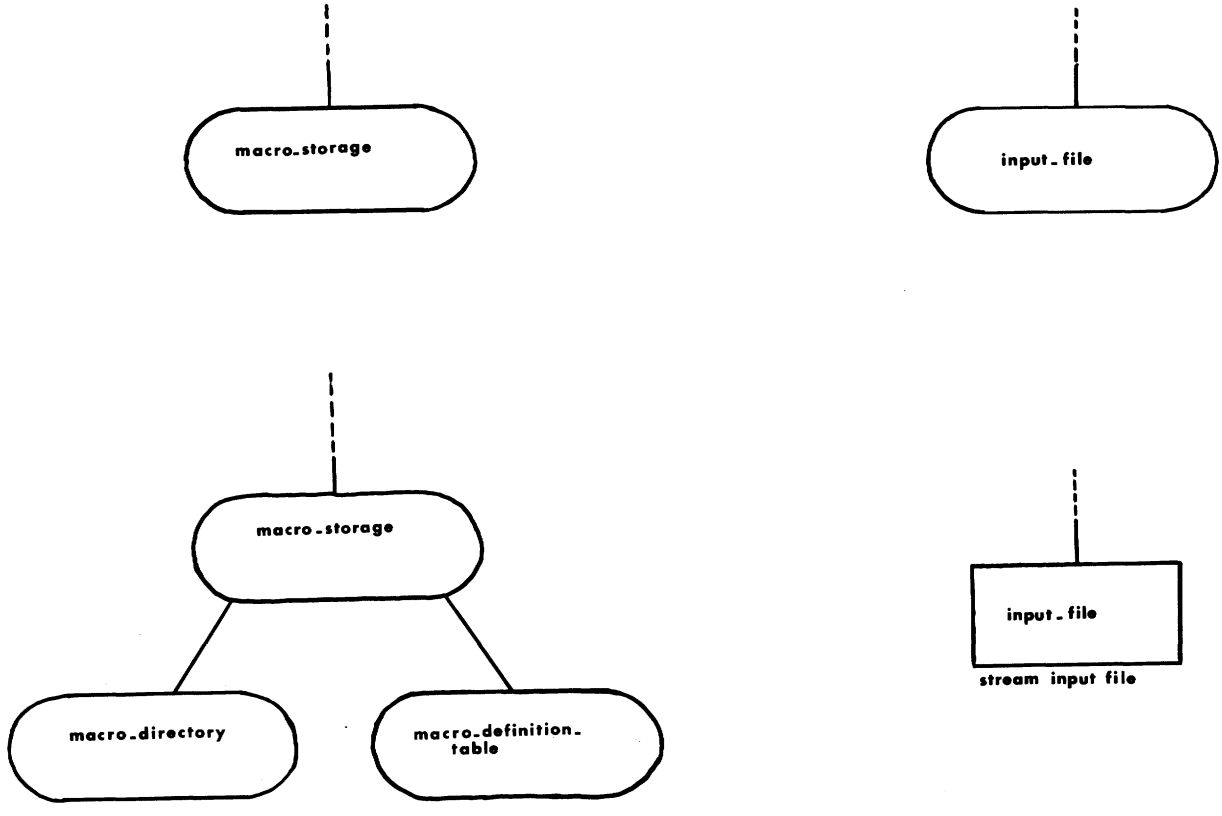

Fig. 2. Refinement of chunk tree nodes.

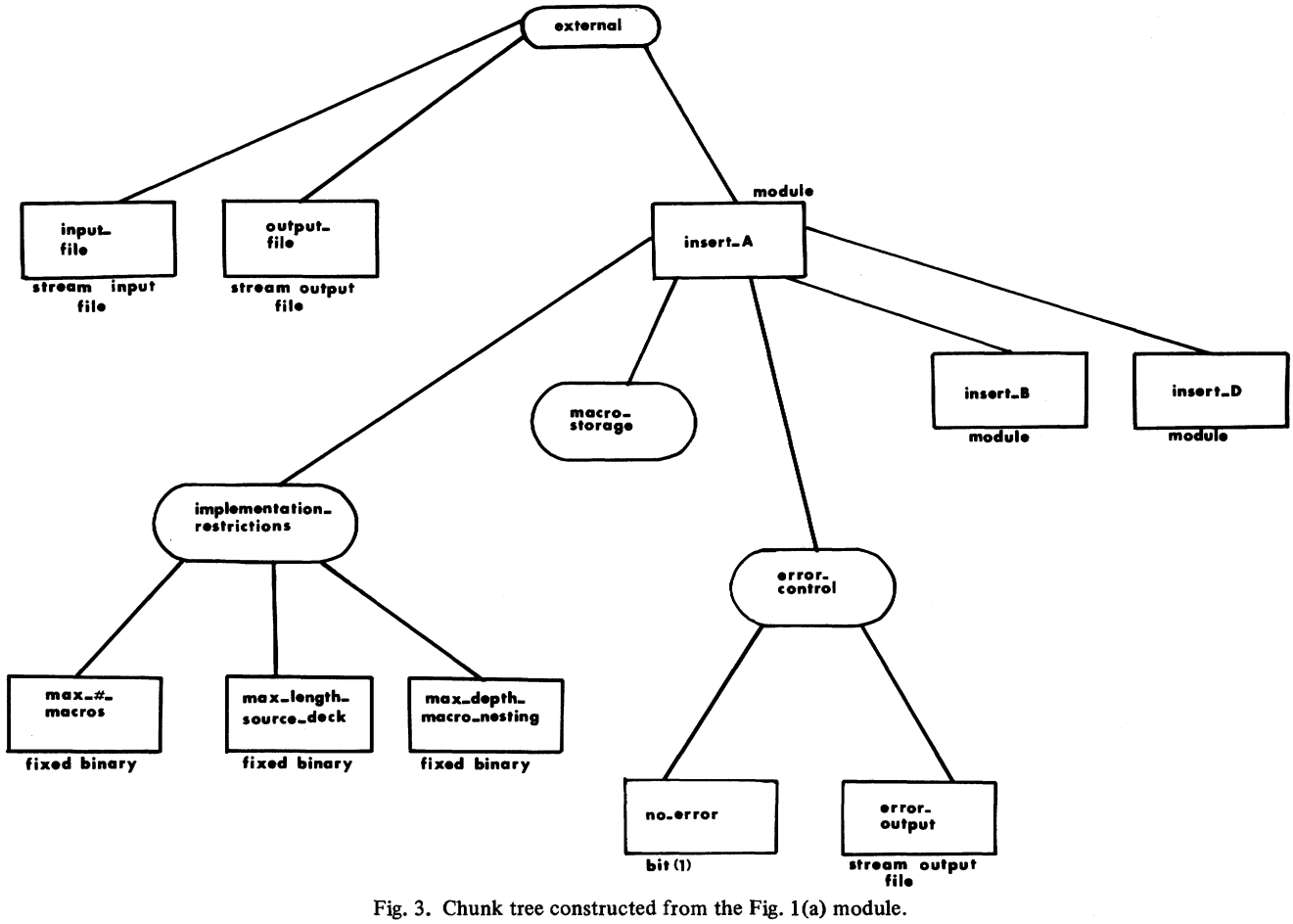

Fig. 3. Chunk tree constructed from the Fig. 1(a) module.

in the module or a submodule rather than passed from the referencing module, depending on the allocation strategy used. The chunks listed in the environment description section, but not passed to the module, are local to the module (length, card_image). The environment of the module consists of the data described by the chunks listed in its environment description section. Some of this data may not yet be refined down to the terminal node level. Only the chunk names listed in the environment description section can be actually referenced within the module.

Replacement of Block Structure by Chunk Structure

Suppose that a programming language were to use chunk structure instead of block structure. Then the objections raised to block structure would be satisfied as follows.

1) Side effects of procedures: all changes to a procedure's external environment would be confined to the part of the external environment passed to the procedure. Side effects would not be possible.

2) Indiscriminate access: chunk structure by itself would not solve this problem, but could contribute to a solution (some suggestions are made later, as concluding remarks). However, denial of access may not be as important for the conscientious programmer as the ability to use a data structure without concern as to how it is (or will be) implemented. Chunk structure would allow the programmer to apply any meaningful operation, including a module as an abstract operation, to the intermediate node representing a data structure without knowing the descendant nodes used in implementing the data structure.

3) Vulnerability: two nodes having the same name will be distinct if the nodes are not direct descendants of the same immediate ancestor node. Since all direct descendants of a node would be visible together on the same page of the chunk tree reference listing, an impending conflict of names could be easily detected and avoided.

4) No overlapping definitions: data could be passed to just those modules that use it, so that modules could access data in an overlapping manner. No module would be forced to have access to data it does not use.

5) The "hole-in-the-scope" problem: there would be no distinct and conflicting static and dynamic scopes for a name. The applicable name would always appear in the environment description section.

6) The inability to reenter a previously exited environment: environments could be freely created and passed to dependent submodules. An environment used in a previously exited module could be passed to a subsequent module.

7) Difficulty in keeping track of environments: a description of the environment would always be visible within the current module. That part of an environment that is passed to dependent submodules would be visible where passed (in the reference title) and where received (in the submodule's heading title).

Implementation Considerations

Chunk structure could be implemented by a preprocessor to an existing compiler or implemented by a compiler or interpreter that could take advantage of the locality of reference inherent in chunk structure. A preprocessor would build the chunk tree from the environment description sections encountered during a first pass, assign unique names to the terminal nodes of the chunk tree, and then replace each chunk name in the code by the corresponding terminal node names during a second pass. Additional code would be generated when intermediate nodes are used as operands. For example, an assignment statement between intermediate nodes of the chunk tree would be replaced by assignments between all of the terminal nodes involved. A reference to a submodule that is to be implemented as a subroutine would be replaced by code for a subroutine call with arguments for all of the terminal nodes being passed.

Although only the terminal nodes of the chunk tree need to be represented at run time, there would be certain advantages if we choose a representation in which intermediate nodes are represented as pointers to their subtrees. The chunk tree would then be replicated in computer storage by a tree of intermediate node pointers and terminal node data, so that the run-time representation would closely resemble the programmer's conceptualization of the data structure of the program. The pointers could be passed to subroutines instead of the (many more) terminal node data implied by the pointers. A "swap" operation between pointers could often be used in place of an assignment operation and would be more efficient since assignment implies copying of all terminal node values. For some parts of a program, upper levels of the chunk tree may be allocated while lower levels are not. Space would then be saved by allocating just the upper level pointers instead of their implied terminal node data.

If this representation is used, then a preprocessor could simply replace each intermediate node chunk name by its corresponding pointer name. Run-time operations for intermediate nodes used as operands would have to be provided, i.e., operations for assignment, test of equality, etc., taking subtree pointers as arguments.

The explicitly listed environments in chunk-structured code can be used by a language processor to compile or interpret more efficiently than by preprocessing. If each module is processed within the context of its own environment description section, then symbol look-ups performed by the processor can be confined to the relatively limited reference space defined by the environment description section. To do this, the language processor uses the chunk tree instead of a symbol table to do its symbol look-ups. The symbol table entries for each name are stored at the chunk tree node for the name. Modules implemented as macros are processed from the module's location in the chunk tree rather than by actually incorporating the macro text into the referencing module's text. If intermediate nodes of the chunk tree are represented as pointers, then the chunk tree need not be constructed prior to code processing, but can grow dynamically as directed by chunk tree refinements described in the environment description sections of modules being processed.

A module's reference region within the chunk tree can be implemented as a list of the pointers that represent those chunk tree nodes found in the environment description section of the module. This list can be stored at the module's location in the

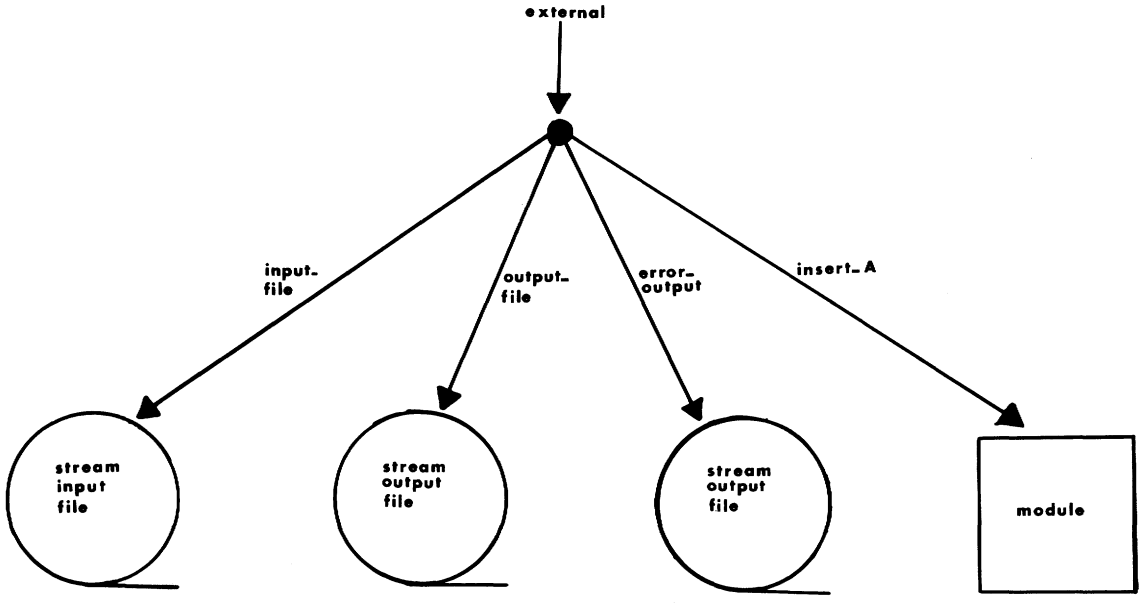

Fig. 4. Implementation of the chunk tree before processing insert_A.

chunk tree. Symbolic references within the module's code can then be resolved by scanning the list. When a module is being processed and a submodule reference is encountered, a list of the pointers that represent the chunk names in the submodule title is passed to the submodule. Note that these names are all in the environment description section of the referencing module, and hence the corresponding pointers are readily found within the limited space of the module's reference region. This list becomes the nucleus for the reference region of the submodule, and can be saved at the submodule's location in the chunk tree until the submodule is itself processed.

Later, when the submodule is processed, its reference region can be constructed from this nucleus list by extending the list to include pointers for all of the nodes that are in the submodule's environment description section. By adding these nodes in a top-down order, each new node will be a direct descendant of a node already in the list. If the chunk tree does not currently contain a node that is listed in the environment description section, then one is allocated and attached at the proper place within the chunk tree. In this way the chunk tree is dynamically refined to conform to the environment description sections encountered during processing. If the environment description section describes a local restructuring of the chunk tree, then the tree is restructured before processing the code for the module and its submodules, and the original shape of the tree is restored after processing.

After construction of its reference region, a module's code can be processed, with all symbol look-ups limited to the module's reference region. If a submodule reference is encountered within this code, then a node for the submodule is attached to the current module's node in the chunk tree.

Fig. 4 suggests an implementation of the externally known part of the chunk tree before processing of the sample program of Fig. 1. Fig. 5 shows the chunk tree as refined by the module insert_A. Note the restructuring involving error_output. All of the pointers except external, insert_B, and insert_D are in insert_A's reference region. In a more developed program, the reference region of any particular module would be smaller in comparison to the entire chunk tree. The pointers labeled "B" will be passed to insert_B as the nucleus of its reference region. Similarly, the pointers labeled "D" will be passed to insert_D.

If the processor is an interpreter, then it processes each submodule at the time that its reference is encountered within the module, so that many modules may be active at one time. The chunk tree locations of active modules form a stack embedded within the chunk tree, with the most recently invoked module at the top of the stack. Upon exit from this module, its referencing module's reference region is restored by "popping" the stack, i.e., by simply moving back to the referencing module's location in the chunk tree.

Processing Time

The restrictions on the length of a module (for example, to fifty lines of code) and on the number of direct descendants of a chunk tree node (to about fifty) can be used to establish an upper bound on the size of reference regions and on the number of look-ups needed to process a module. This will then set a bound on the time required to process a module.

Processing of a module begins with the construction of its reference region, which involves look-ups of direct descendants in the chunk tree. The number of such look-ups is limited by the size of the environment description section, which is no larger than the size of the module, and hence is bounded above by a number that can be determined from the module length limit. An upper bound on the time required to do a single such look-up is determined by the limit on the number of direct descendants of a chunk tree node, which limits the number of comparisons needed in the search. Thus the total look-up time needed for construction of a module's reference region is bounded.

After the reference region is constructed, the module's code is processed. The number of symbol look-ups needed to pro-

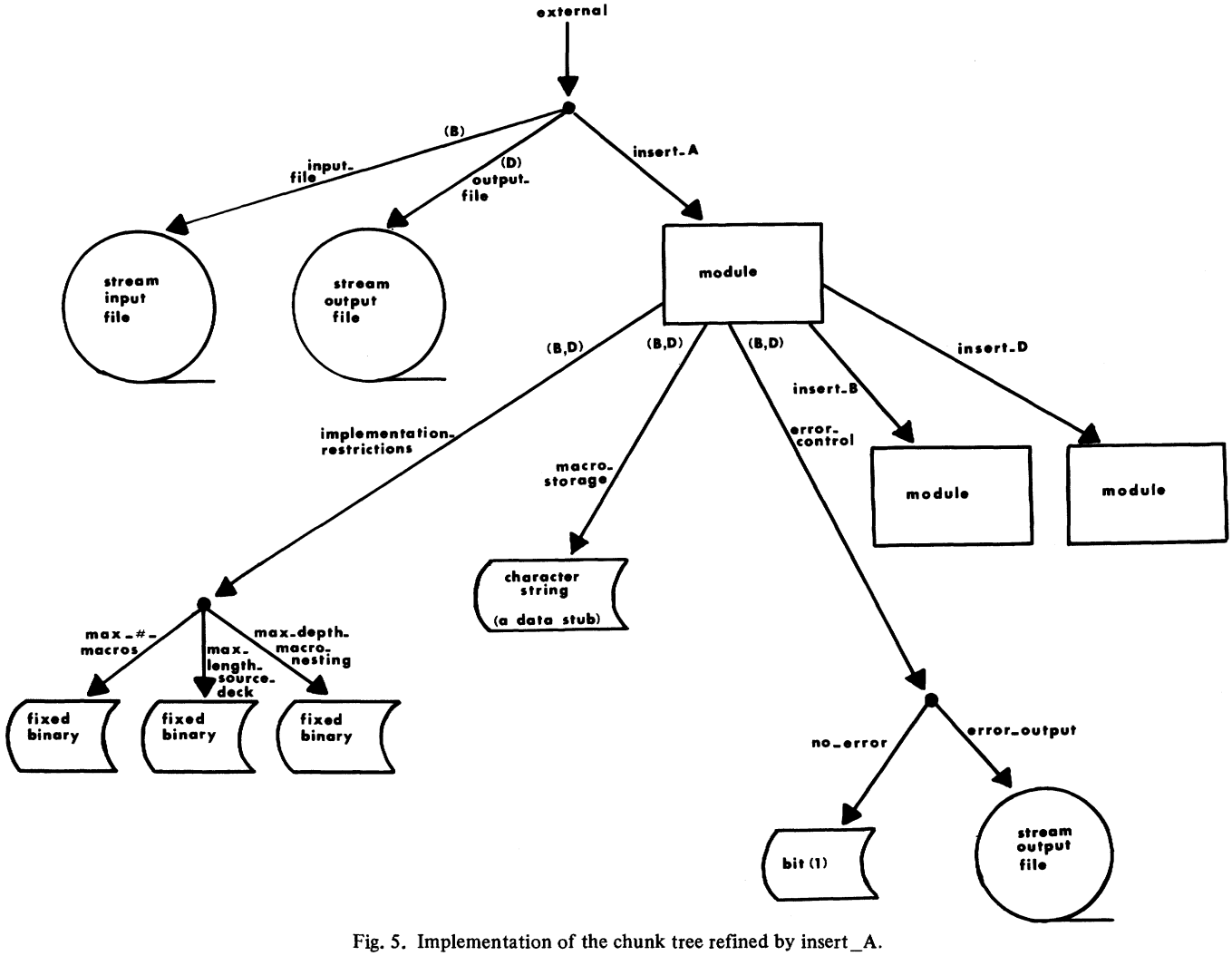

Fig. 5. Implementation of the chunk tree refined by insert_A.

cess the code is bounded by a number determined by the module length limit. A bound on the time required for each individual look-up is determined by the maximum size of the reference region, which also results from the module length limit. Thus the time used for all symbol look-ups while processing a module is bounded. All other processing activities require at most a given amount of time per fifty lines of code, and hence the total time required to process a module of chunk-structured code has a fixed upper bound.

Let N be the length of a program, as measured by the number of lines of code. N is proportional to the number of modules in a program due to the module length limit. A compiler processes each module only once, and thus the time required to compile a chunk-structured program will increase linearly with N. Compilation can also be performed in linear time using hashing techniques for symbol table construction and look-up [18]. However, in order to limit the load factor hash tables require a good estimate of the number of symbols in the program before compilation begins. Otherwise, additional time is spent on slower methods of collision resolution or in rehashing to a larger table. By contrast, the chunk tree can grow dynamically while compiling the program, and with no loss of efficiency as the tree becomes larger. Balanced trees used as symbol tables [18] can also grow dynamically while compiling a program. However, look-up time within a balanced tree is proportional to log N (assuming that symbols occur with constant density throughout the program). Computation time using balanced trees will then be proportional to N*log N.

Locality of Reference

Locality of reference, i.e., the tendency of a program to reference only a limited subset of its code and data during a limited interval of time, can be used to reduce the size of working sets of pages in a virtual memory system [19]. Chunk structure suggests a strategy for analyzing a program prior to run time to determine a composition of pages that takes advantage of the program's locality of reference. For each module, this strategy would attempt to keep its code and the data listed in its environment description section together within the same pages. This code and data (and no other code and data) will be referenced by the module. To the extent that the

code and data fits into a small number of pages, these pages will constitute a small working set whenever the module (but no submodule) is executing. If, additionally, these pages tend to hold code and data belonging to the same subtrees of the chunk tree, then the working set will tend to remain stable or decrease as submodules are being executed.

Chunk structure could also provide a kind of locality of reference for the programmer and for the language processor (compiler or interpreter). We will understand this to mean that when programmer or language processor must look up a data reference the look-up is always performed within a limited region of the total reference space. Look-ups performed within this limited region can be fast in comparison with look-ups over the entire reference space, thus decreasing the effort needed for programming, compilation, or interpretation. We have previously seen that a language processor for chunk-structured code can be implemented so as to limit its symbol look-ups to the current module's reference region within the chunk tree. The programmer developing a new module references existing data names in order to be reminded of attributes, spellings, and relationships to other data. The chunk tree reference listing will provide this information (much as an attribute and cross-reference listing does now) except that related chunks will be located physically near one another, while unrelated chunks that happen to have the same name will be physically separated. The programmer's references within the listing can be limited to subtrees of nodes passed to the current working module. Once the environment description section of the current working module has been established, the programmer's references can be limited to this local description of the module's environment.

Chunks as Data Abstractions

Each intermediate node of the chunk tree can be regarded as a data abstraction, with its descendant nodes describing the data used in implementing it. The data abstraction can be refined into its implementation data in small design steps governed by the requirements of the modules as they are being coded, so that premature design decisions are not forced on the programmer. To avoid design errors, the programmer should know all abstract operations (modules) that will access the implementation data of a data structure to be refined, so that all effects of the design decision can be considered in advance. Dijkstra accomplishes this by collecting and refining all such accessing modules simultaneously with the data structure within the same "pearl" [20]. McGowan and Kelley [14] recommend a separate design phase for discovering all operations on data structures, so that data structures can be properly designed before the actual coding begins. Liskov and Zilles [21] define a data structure by the "cluster" of all operations that access it, and hence the cluster itself gives the desired list of operations relevant to the implementation of the data structure. If chunk structure is used, then the desired list of operations consists of those sentences from titles of uncoded modules that contain the chunk name of the data structure to be refined. Since chunk marks identify chunk names within titles, these sentences could be easily located and listed by a programming aid.

Fig. 6(a) shows how a chunk-structured version of a sample program of Dijkstra might appear just prior to the design decision on refinement of the data structure "image" (cf. [20, pp. 50-59]). The uncoded modules whose titles mention "image" are "Print the ←image.", "Clear the image←.", and "Mark the position at ←x_coord, ←y_coord in ←image←.". Note that these are the same modules that Dijkstra collects together for refinement with "image" in the pearl LINER [Fig. 6(b)]. Chunk-structured versions of a design segment and of a program with clusters are given in [16].

Debugging

Desk checking, if done at all, is often performed in a passive manner. Program code always seems to "look good" to its author. For chunk-structured code, each environment description section provides the programmer with a convenient and moderately sized check list of variables whose values must be set correctly before exiting the module. This should encourage a more active desk check. The requirement that all passed chunks be explicitly stated in the module title compels the programmer to write titles that more completely describe the effects of a module. This aids desk checking since the semantic part of the check will then be based on fewer implicit assumptions. The chunk marks can also be used to help detect some obvious errors, such as changing the output value of input-only data or failing to initialize output-only data.

In the approach to top-down debugging described by Mills [12] and by Baker [22], uncoded modules are macro-replaced by dummy module "stubs" to produce compilable code that can be run and debugged. In a similar way, chunk-structured code can be compiled and run for debugging purposes even though it contains unimplemented data abstractions. These data abstractions can be given a dummy character string type and assigned dummy character string values at run time by those stubs or modules that output the data abstraction. These string values might identify the data abstraction, the stub or module "author" of its current value, and a serial number for the value assigned by the author. Operations on the data abstraction then become operations on the dummy type, with values that can be printed as an aid to debugging.

Psychological Chunks

Psychologists have determined that the human "short-term memory," which is used extensively in problem solving, can effectively hold only a limited number of items at one time. This number has been estimated at around seven [23] or five [24], while four is an optimal value in one mathematical model [25]. In order to manage a larger number of items, the human mind can learn to perceive a related group of items as an individual "chunk," so that (after learning) the number of chunks is again reduced to around seven or five. Miller [23] describes the process as follows:

> A man just beginning to learn radio-telegraphic code hears each *dit* and *dah* as a separate chunk. Soon he is

```
COMPFIRST
begin "Draw the curve <fx,<fy on the printer."
env
<fx
<fy
/image
end env;

"Build an image< of curve <fx,<fy.";
"Print the <image."
end


CLEARFIRST
begin "Build an image< of curve <fx,<fy."
env
<fx
<fy
image<
end env;

"Clear the image<.";
"Set marks onto the <image< corresponding to <fx,<fy."
end


ISCANNER
begin "Set marks onto the <image< corresponding to <fx,<fy."
env
<fx
<fy
<image<
/i integer
end env;

i:=0;
while i<1000 do {"Add mark number <i of curve <fx,<fy to the <image<."; i:=i+1}
end


COMPPOS
begin "Add mark number <i of curve <fx,<fy to the <image<."
env
<fx integer procedure
<fy integer procedure
<image<
<i integer
/x_coord integer
/y_coord integer
end env;

x_coord:=fx(i); y_coord:=fy(i);
"Mark the position at <x_coord, <y_coord in <image<."
end
```

(a)

Fig. 6(a). Chunk structured version of Dijkstra's sample program "Draw."

able to organize these sounds into letters and then he can deal with the letters as chunks. Then the letters organize themselves as words, which are still larger chunks, and he begins to hear whole phrases. . . . There are many ways to do this recoding, but probably the simplest is to group the input events, apply a new name to the group, and then remember the new name rather than the original input events.

Psychological chunking has inspired several proposed programming practices intended to keep the mental requirements of programming down to intellectually manageable proportions [26]-[29]. The convention described here should be regarded as another such proposal. The intermediate nodes of the chunk tree conform quite closely to the psychological notion of a "chunk," i.e., they collect items together into a named group that permits the programmer to think of and manipulate data in terms of fewer units. We therefore feel that our use of "chunk" terminology is reasonable.

Miller describes the formation of chunks as a bottom-up process, as is appropriate for organizing perceptions, i.e., from the concrete to the abstract. By contrast, coding of chunks is top-downwards, from the abstract to the concrete. Presumably these programming abstractions are based on prior experience and education, or on simple confidence that an implementation can eventually be designed.

Concluding Remarks and Suggestions

We have described a convention which uses chunks for grouping and naming data and passing named groups to program modules. The convention can be applied to the problems associated with global variables and can also provide a mechanism for top-down refinement of data. Programs that follow the convention will have a locality of reference that permits compilation in linear time without using hash tables. Programming should be simplified by use of the convention since the chunk tree organizes data in a manner convenient to the programmer

```
LINER

begin "Print the <image."
env
<image
     line array[0:49]
/j integer
end env;

j:=49;
while j>0 do {"Print <line[j]."; j:=j-1}
end


begin "Clear the image<."
env
image<
     line array[0:49]
/j integer
end env;

j:=49;
while j>0 do {"Clear line[j]<."; j:=j-1}
end


begin "Mark the position at <x_coord, <y_coord in <image<."
env
<image<
     line array[0:49]
<x_coord integer
<y_coord integer
end env;

"Mark <line[y_coord]< at position <x_coord."
end
```

(b)

Fig. 6(b). Refinement of data in chunk structured version of Dijkstra's sample program "Draw."

and allows the programmer to completely describe and see the current working environment. We feel that the data abstraction capability provided by chunk structure is easier for the programmer to use than those provided in CLU [21], [30] or in Pascal [31] since types do not have to be defined for each abstraction. However, chunk structure does not provide for protection of abstractions as does CLU. The actual value to the programmer of chunk structure (or other new language elements) can best be determined by human factors studies [29], [32].

Chunk structure has the disadvantage of requiring the programmer to list each node repeatedly, in every module that uses it. A text-editing system could help by copying existing parts of the chunk tree to a new module. (One can also imagine a more elaborate programming aid, along the lines of Pearl [33], that could further simplify the task.) We feel that the extra effort involved is more than compensated for by the gain in program clarity. Chunk structure also requires more text than block structure, due to the presence of the environment description section and the general wordiness of the module titles. The space remaining for executable code seems too confining in some cases, suggesting that a somewhat larger module length limit may be necessary.

Chunk structure could facilitate a solution to the problem of indiscriminate access by making it easy to specify an access-denying attribute for a whole group of data by assigning the attribute to the group's single ancestor node in the chunk tree. A different strategy for denying access to the data that implements a data structure would be to pass the implementation data to the modules that implement the data structure and then pass the modules, but not the data, to those modules that use the data structure. Thus if braces in the title delineate chunks that are to be passed to the module immediately, then

```
env;
...
/stack_A
/stack_A_cluster
     push_A entry ("Push ←x onto {←stack_A←}.")
     pop_A entry("Pop x← from {←stack_A←}.")
...
end env;
```

would pass stack_A immediately to push_A and pop_A, thus creating a stack cluster for stack_A [21]. The chunk marks within braces should then be omitted from subsequent title references since their chunks have already been passed, e.g., "Push ←x onto stack_A.". Thus there is no confusion between what has been passed to the module and what remains to be passed.

We have attempted to present a nearly minimal convention for the use of chunk structure in programming languages. The convention could be usefully extended by allowing the programmer to define commonly recurring subtrees of the chunk tree as new terminal node types and by allowing data refinement in terms of Hoare's data structuring methods [20], [34]. For this purpose the indentation notation for trees can be augmented to indicate the structuring method of an intermediate node of the chunk tree, as suggested by [35].

We have not explored the effects of chunk structure on program verification [36], but we note that intermediate nodes of the chunk tree can be mentioned in assertions and appear in verification lemmas (verification conditions), thus possibly

facilitating verification of module properties in terms of the abstract data actually referenced by the module. Assertions could also be attached to a chunk tree node to describe the invariant conditions that hold in the data structure described by the node [37]-[39]. For example, if a node is to represent an ordered list, then an assertion for it might say that each pair of successive entries in the list is ordered. The assertion should then be true whenever a module has access at or above the node, but need not be true while access is within the subtree of the node (e.g., while an entry is being inserted or deleted from the list). Verification would consist of showing that the assertion is initially true and that for any module that accesses the subtree of the node, if the assertion is true before module entry, then it will be true on module exit.

Acknowledgment

This paper has benefitted from comments by, and discussions with, R. J. Baron, M. J. Broussard, M. U. Farooq, E. Horowitz, W. P. Johnson, E. P. Katz, T. G. Lewis, R. E. Michelsen, N. Ostrich, L. Ragland, D. J. Romaguera, B. Shneiderman, J. E. Urban, T. M. Walker, and J. A. Winkler. The reviewer's comments have also improved the clarity of the presentation.

References

[1] R. B. Kieburtz, "Steps toward verifiable programs," Dep. Comput. Sci., State University of New York at Stony Brook, Tech. Rep. 12, Nov. 1972.
[2] W. Wulf and M. Shaw, "Global variable considered harmful," *Sigplan Notices*, vol. 8, pp. 28-34, Feb. 1973.
[3] J. Earley, "Naming structure and modularity in programming languages," Dept. Comput. Sci., Univ. of California, Berkeley, Tech. Rep. 17, Aug. 1973.
[4] J. E. George and G. R. Sager, "Variables–Bindings and protection," *Sigplan Notices*, vol. 8, pp. 18-29, Dec. 1973.
[5] L. Presser and J. R. White, "Making global variables beneficial," in *IFIP Conf. Proc.* Amsterdam, The Netherlands: North-Holland, 1974, pp. 413-418.
[6] D. L. Parnas, "On the need for fewer restrictions in changing compile-time environments," *Sigplan Notices*, vol. 10, pp. 29-36, May 1975.
[7] E. W. Dijkstra, "An essay on the notion: The scope of variables," in *A Discipline of Programming.* Englewood Cliffs, NJ: Prentice-Hall, 1976.
[8] P. A. Carr, "Name management in the construction of large programs," Ph.D. dissertation, Dep. Comput. Sci., Iowa State Univ., Aug. 1976.
[9] P. Naur, "Answers to comments on Algol 60," *Commun. Ass. Comput. Mach.*, vol. 4, pp. A16-A17, Feb. 1961.
[10] D. E. Knuth and J. N. Merner, "Algol 60 confidential," *Commun. Ass. Comput. Mach.*, vol. 4, pp. 268-272, June 1961.
[11] R. Conway and D. Gries, *An Introduction to Programming*, 2nd. ed. Cambridge, MA: Winthrop, 1975, section III 1.3, pp. 142-143.
[12] H. D. Mills, "Top-down programming in large systems," in *Debugging Techniques in Large Systems*, R. Rustin, ed. Englewood Cliffs, NJ: Prentice-Hall, 1971.
[13] —, "Mathematical foundations for structured programming," IBM Corp., Gaithersburg, MD, FSC 72-6012, 1972.
[14] C. L. McGowan and J. R. Kelley, *Top-Down Structured Programming Techniques.* New York: Mason/Charter, 1975.
[15] D. E. Knuth, *The Art of Computer Programming*, vol. 1. Reading, MA: Addison-Wesley, 1969, pp. 309-310.
[16] E. Towster, "Chunk structure–A convention for explicit declaration of environments and top-down refinement of data," Dept. Comput. Sci., Univ. Southwestern Louisiana, Lafayette, LA, Tech. Rep. 78-5-3, 1978.
[17] J. J. Donovan, *Systems Programming.* New York: McGraw-Hill, 1972.
[18] D. E. Knuth, *The Art of Computer Programming*, vol. 3. Reading, MA: Addison-Wesley, 1973.
[19] P. J. Denning, "Virtual memory," *Computing Surveys*, vol. 2, pp. 153-189, Sept. 1970.
[20] O.-J. Dahl, E. W. Dijkstra, and C. A. R. Hoare, *Structured Programming.* New York: Academic, 1972.
[21] B. Liskov and S. Zilles, "Programming with abstract data types," *Sigplan Notices*, vol. 9, pp. 50-59, Apr. 1974.
[22] F. T. Baker, "Chief programmer team management of production programming," *IBM Syst. J.*, vol. 11, pp. 56-73, 1972.
[23] G. A. Miller, "The magical number seven, plus or minus two: Some limits on our capacity for processing information," *Psychological Review*, vol. 63, pp. 81-96, Mar. 1956; reprinted in G. A. Miller, *The Psychology of Communication.* New York: Basic Books, 1967.
[24] H. A. Simon, "How big is a chunk?" *Science*, vol. 183, pp. 482-488, Feb. 8, 1974.
[25] D. K. Dirlam, "Most efficient chunk sizes," *Cognitive Psychology*, vol. 3, pp. 355-359, 1972.
[26] G. M. Weinberg, *PL/I Programming: A Manual of Style.* New York: McGraw-Hill, 1970, pp. 239-240.
[27] —, *The Psychology of Computer Programming.* New York: Van Nostrand, 1971, pp. 224-229.
[28] D. Frost, "Psychology and program design," *Datamation*, vol. 21, pp. 137-138, May 1975.
[29] B. Shneiderman, "Exploratory experiments in programmer behavior," *Int. J. Comput. Information Sciences*, vol. 5, no. 2, pp. 123-143, 1976.
[30] J. B. Dennis, "An example of programming with abstract data types," *Sigplan Notices*, vol. 10, no. 7, pp. 25-29, July 1975.
[31] K. Jensen and N. Wirth, *PASCAL User Manual and Report.* Berlin: Springer-Verlag, 1974.
[32] B. Shneiderman, "Experimental testing in programming languages, stylistic considerations and design techniques," in *Proc. Nat. Comput. Conf.*, 1975, pp. 653-656.
[33] R. A. Snowdon, "Pearl–A system for the preparation and validation of structured programs," in W. C. Hetzel, Ed., *Program Test Methods.* Englewood Cliffs, NJ: Prentice-Hall, 1973.
[34] C. A. R. Hoare, "Data reliability," in *Proc. Int. Conf. on Reliable Software*, *Sigplan Notices*, vol. 10, pp. 528-533, June 1975.
[35] J. E. Urban, "A specification language and its processor," Ph.D. dissertation, Dep. Comput. Sci., Univ. of Southwestern Louisiana, Lafayette, LA, Tech. Rep. 78-5-1, 1978.
[36] R. L. London, "A view of program verification," in *Proc. Int. Conf. on Reliable Software*, *Sigplan Notices*, vol. 10, pp. 534-545, June 1975.
[37] M. Foley and C. A. R. Hoare, "Proof of a recursive program: Quicksort," *The Computer J.*, vol. 14, pp. 391-395, Nov. 1971.
[38] M. S. Laventhal, "Verification of programs operating on structured data," Project MAC, M.I.T., Cambridge, MA, MAC TR-124, Mar. 1974.
[39] C. A. R. Hoare, "Proof of correctness of data representations," *Acta Informatica*, vol. 1, pp. 271-281, 1972.

Edwin Towster (S'65-M'70) was born in Chicago, IL, in 1933. He received the B.S. in mathematics from the University of California, Berkeley, in 1955 and the M.S. and Ph.D. degrees in computer science from the University of Wisconsin, Madison, in 1966 and 1970.

He began programming in 1958 at the Lawrence Berkeley Laboratory, working on an IBM 650 computer. He has also held positions as a member of the Technical Staff at the Mitre Corporation, Bedford, MA, and as an Assistant Professor of Computer Science at the University of Iowa. He is currently an Associate Professor of Computer Science at the University of Southwestern Louisiana, Lafayette. His interests include reliable programming methods, automatic programming, computer concept formation, and trainable software.